Informe resumen: Sistemas Distribuidos

Oct 27, 2026 · @Pilar

Este informe condensa el apunte de la materia clase por clase. Los temas que el apunte repite en varias clases (Lamport, Raft, CAP, quórums, outbox) se desarrollan una sola vez y en el resto se remite a esa sección.

Clase 01 — Fundamentos: pensar bajo incertidumbre

Idea central: en un sistema distribuido un observador remoto no puede distinguir entre una falla real del destino, una falla de comunicación y una demora larga. El problema es epistemológico: el sistema no da conocimiento perfecto de su propio estado. Por eso el objetivo no es adivinar el estado real, sino garantizar un comportamiento razonable con información incompleta.

Definición

Un sistema distribuido es un conjunto de procesos o nodos que se ejecutan en máquinas distintas, se comunican por mensajes y cooperan para que el usuario perciba un solo sistema. Consecuencias: la comunicación no es instantánea, las fallas son parciales y el conocimiento del estado global es limitado. Un problema se vuelve distribuido cuando su corrección depende de coordinar componentes que no comparten memoria, reloj ni observación perfecta.

Modelo síncrono vs. asíncrono

Modelo	Cotas de red y cómputo	Consecuencia
Síncrono	Conocidas	Una demora fuera de cota puede interpretarse como falla
Asíncrono	No hay cotas confiables	Una demora arbitrariamente larga es indistinguible de una falla

Métodos de trabajo

- **Relevar, analizar, proponer.** Relevar = síntomas observables (lo que efectivamente pasó), contexto, restricciones y datos faltantes. Analizar = causas compatibles, evidencia e hipótesis descartadas. Proponer = garantía buscada, solución elegida y

trade-off aceptado, con argumentos. Primero se separan las tres dimensiones para no mezclar observación, inferencia y decisión; después se conectan para justificar la propuesta.

- **Framework D = (P, F, G, S, T):** Problema, Fallas asumidas, Garantía requerida, Solución elegida, Trade-off aceptado. Pensar distribuidamente es saber qué información no tenés, qué falla modelás, qué garantía necesitás y qué costo aceptás.

CAP (primera aproximación)

Con estado replicado no se puede garantizar a la vez Consistencia (todas las respuestas ven una única versión correcta), Disponibilidad (toda operación recibe respuesta) y Tolerancia a particiones. Las particiones son un hecho de la vida, así que la decisión real es, ante partición, rechazar operaciones para preservar corrección o responder igual aceptando inconsistencias. Desarrollo completo en la Clase 08.

Errores conceptuales frecuentes

- Creer que timeout implica crash.
- Creer que la red es confiable salvo casos extremos.
- Asumir una visión global perfecta del sistema.
- Usar CAP como slogan y no como restricción ante partición.

Para recordar: no hay solución perfecta; hay que explicitar supuestos, elegir garantías y defender el trade-off.

Clase 02 — RPC, timeouts, retries e idempotencia

Pregunta central: si todo ocurre por mensajes sobre una red incierta y sin garantía de entrega, ¿qué significa que “la operación ocurrió”?

Una llamada remota atraviesa capas

Aplicación → RPC → transporte → red → servidor. Cada capa puede reaccionar distinto ante una falla: una reintenta mientras otra ya venció su timeout, una oculta la pérdida y otra solo ve demora, y el servidor puede ejecutar aunque el cliente ya haya abandonado.

RPC

Mecanismo para ejecutar una función de otra máquina como si fuera local. Intenta ocultar la red, pero **no elimina la incertidumbre**: en un sistema robusto la red se gestiona, no se oculta. Ante falta de respuesta no se sabe si se perdió el request, se perdió la respuesta, el servidor está lento o está caído.

Timeout, retry e idempotencia

- **Timeout:** el cliente decide esperar, reintentar o informar error. Solo indica que no hubo respuesta dentro de una ventana; **no implica falla**.
- **Retry:** recupera fallas transitorias, pero puede duplicar la operación y agrega presión sobre el sistema.
- **Idempotencia:** aplicar la operación varias veces produce el mismo resultado observable. Es lo que hace seguro reintentar.

Semánticas de entrega

Semántica	Evita	Riesgo
At-most-once	Duplicación	Puede perder la ejecución
At-least-once	Pérdida	Puede duplicar
Exactly-once	Ambas	Garantía lógica costosa (identidad + idempotencia), no magia de la red

Trade-offs: confiabilidad vs. complejidad, latencia vs. corrección, retry vs. duplicación, idempotencia vs. más estado.

Para recordar: comunicar es decidir bajo incertidumbre y reconstruir qué pudo haber ocurrido en cada capa.

Clase 03 — Replicación, consistencia y modelos de estado

Pregunta central: ¿cómo mantenemos estado cuando hay múltiples copias? Replicar = varios nodos mantienen el mismo estado lógico, para ganar disponibilidad, latencia (lecturas cercanas) y tolerancia a fallas.

Hecho estructural: sin coordinación perfecta, las réplicas divergen (latencias variables, pérdidas y reintentos, escrituras concurrentes, particiones). La divergencia no es un bug: es consecuencia del modelo.

Modelos de consistencia

Un modelo de consistencia define **qué valores pueden observar los clientes y en qué orden** (lo que observan, no lo que hay guardado).

- **Linearizabilidad (fuerte):** se comporta como una sola copia y respeta el orden de tiempo real. Si una operación terminó antes de que otra empiece, ese orden se respeta; si se superponen, se puede elegir un orden coherente con una única copia. Es lenta y

poco tolerante a particiones.

- **Consistencia causal:** respeta las dependencias causales (happens-before) sin imponer orden total.
- **Consistencia eventual:** si dejan de llegar escrituras, las réplicas convergen; no hay garantía inmediata de lectura. Ejemplo: DNS.

Trade-off: fuerte = coordinación + latencia + menor disponibilidad; débil = disponibilidad + anomalías temporales.

Causalidad y relojes lógicos

El tiempo físico no sirve para ordenar: los relojes derivan o se corrigen, la red reordena mensajes y "una hora menor" no prueba causalidad. La pregunta correcta es qué evento pudo influir sobre cuál.

Happens-before (a → b) si: (1) están en el mismo proceso y a ocurre antes; (2) a es el envío de un mensaje y b su recepción; (3) por transitividad. Dos eventos son **concurrentes (a ‖ b)** si no hay relación en ninguna dirección; no significa simultáneos.

Relojes de Lamport. Cada proceso tiene un contador L (inicia en 0, no representa tiempo real):

1. Evento local: L ← L + 1.
2. Envío: incrementa y adjunta L al mensaje.
3. Recepción: L ← max(L, t) + 1, para que la recepción quede por encima del envío.

Garantía: a → b ⇒ L(a) < L(b). **Límite:** L(a) < L(b) no prueba a → b; no distingue causalidad de concurrencia. Con un desempate (ID de proceso) da un orden total convencional.

Relojes vectoriales. Cada proceso mantiene un vector con una posición por proceso; [2, 0, 1] = conoce causalmente 2 eventos del primero, 0 del segundo y 1 del tercero.

1. Evento local: incrementa solo su componente.
2. Envío: adjunta su vector actualizado.
3. Recepción: máximo componente a componente y luego incrementa su propia componente.

Comparación: Va < Vb si todos los componentes de Va son ⩽ y al menos uno es <. Si Vb domina a Va, Vb es causalmente posterior; si ninguno domina, son **concurrentes** (conflicto potencial). Los vectoriales eliminan la ambigüedad de Lamport, pero ninguno de los dos **decide** qué hacer con el conflicto.

Arquitecturas de replicación

Arquitectura	Cómo funciona	Ventaja	Desventaja
Leader-based	Un líder serializa escrituras; followers replican de forma síncrona (el líder espera confirmaciones: más latencia) o asíncrona (confirma antes: réplicas atrasadas)	Orden común simple	Si cae el líder, las escrituras se frenan hasta recuperarlo o elegir otro
Multi-leader	Varios líderes aceptan escrituras e intercambian actualizaciones	Escrituras locales rápidas	Conflictos inevitables que hay que detectar y resolver
Leaderless (Dynamo)	N réplicas; escribir en W, leer de R; cliente más pesado	Sin líder central	Hay que elegir la versión más nueva (vector clocks) y resolver conflictos

Quórum clásico: R + W > N. Garantiza que los conjuntos de lectura y escritura compartan al menos una réplica (ej.: N = 5, W = 3, R = 3). No garantiza consistencia fuerte: ver Clase 08.

Resolución de conflictos

Estrategia	Idea	Pro	Contra
LWW (last-write-wins)	Gana el timestamp mayor	Simple	Puede descartar cambios válidos; el timestamp mayor no siempre es la escritura real más tardía
Merge semántico	Combina según el significado del dato (reglas de negocio)	Preserva ambos cambios	No todo dato se puede combinar
Versionado (vector clocks)	Detecta si las versiones son causales o concurrentes	Identifica el conflicto con precisión	No lo resuelve: requiere una política aparte

Anti-entropy

En vez de asumir que toda actualización llega bien, el sistema compara y repara diferencias después.

- **Gossip:** cada nodo comparte periódicamente su estado con algunos otros y los datos se propagan. Descentralizado, pero la propagación tarda y no es predecible.
- **Read-repair:** al leer, si las réplicas devuelven valores distintos, se repara la desactualizada. Las lecturas ayudan a converger.

Para recordar: replicar estado obliga a elegir qué inconsistencias aceptar y cuándo.

Clase 04 — Consenso: FLP, Raft y Paxos

Objetivo: que nodos en máquinas distintas, con posibles fallas, lleguen a una decisión compatible. Consenso impone un orden global a costa de latencia y disponibilidad. (Esta sección integra también lo que la Clase 08 agrega sobre Raft.)

Propiedades del consenso

- **Agreement:** dos procesos correctos no deciden valores distintos.
- **Validity:** el valor decidido es válido; en la formulación habitual, uno de los propuestos.
- **Termination:** los procesos correctos terminan decidiendo.

Se agrupan en **safety** (nunca pasa algo incorrecto, p. ej. no vender dos veces la misma entrada aunque haya crashes o particiones) y **liveness** (el sistema logra avanzar y eventualmente decide).

FLP

No es un algoritmo sino un resultado de imposibilidad: en un sistema **totalmente asíncrono** donde al menos un proceso puede fallar por crash, **ningún algoritmo determinista garantiza terminar siempre** el consenso manteniendo safety. La intuición: existen **estados bivalentes** (todavía se puede decidir cualquiera de dos valores) y un **scheduler adversario** que ordena pasos y mensajes puede mantener al sistema en una **ejecución infinita** sin decidir. Por eso los algoritmos reales agregan supuestos: timeouts, sincronía parcial.

Máquina de estados replicada

Una máquina de estados aplica (estado, comando) → (nuevo estado, respuesta) de forma determinista. Si dos réplicas aplican los mismos comandos en distinto orden, terminan distintas. Solución: cada réplica tiene una copia del **log de comandos**, el mismo código

determinista y su estado local; si aplican los mismos comandos comprometidos en el mismo orden, convergen. La pregunta clave es quién decide qué entra al log y en qué orden: eso es el consenso.

Total-order broadcast: todas las réplicas correctas entregan los mismos mensajes en el mismo orden. Lamport no alcanza, porque numerar eventos no es acordar decisiones. Consenso elige un valor por posición; repetirlo para las posiciones 87, 88, 89… construye el log ordenado.

Raft

Algoritmo de consenso con líder que mantiene un log replicado y consistente.

Roles: Follower (sigue al líder y vota), Candidate (pide votos), Leader (recibe propuestas y replica el log).

Conceptos: term = número monótono creciente que identifica cada ronda de liderazgo y sirve para rechazar autoridades obsoletas (un nodo en term 13 rechaza mensajes de term 12); **index** = posición de una entrada en el log; **log entry** = término + comando en un índice.

Elección de líder:

1. El líder envía heartbeats periódicos.
2. Si un follower no los recibe y vence su election timeout, pasa a candidate.
3. Incrementa su term y pide votos.
4. Con mayoría (y cumpliendo las reglas de elección) se vuelve líder.

Replicación (N = 5, mayoría = ⌊N/2⌋ + 1 = 3):

1. El cliente envía la operación al líder, que la agrega a su log (1/5, no confirmada).
2. Los followers la persisten y responden (2/5, todavía no).
3. Con 3/5 el líder avanza el commitIndex (la entrada debe ser de su término actual).
4. El líder aplica, responde al cliente y comunica leaderCommit; los followers aplican al enterarse.

Estado de una entrada	Significado
Almacenada / replicated	Está en el log de una o más réplicas
Committed	Raft garantiza que no será reemplazada por otra incompatible en esa posición
Applied	La máquina de estados ejecutó el comando
Confirmed	El cliente recibió la confirmación

Si cae el líder: si cae antes de la mayoría, la entrada no estaba committed y el cliente no debía tener confirmación. Si cae después, la decisión se preserva: toda nueva mayoría se intersecta con la anterior y dos reglas lo garantizan:

- **Log matching:** si dos logs tienen una entrada con el mismo índice y término, esa entrada y todas las anteriores coinciden.
- **Leader completeness:** una entrada committed en un término está en los logs de todos los líderes posteriores.

Costo: necesita mayoría para avanzar. Si un cluster de 5 pierde contacto con 3, los 2 restantes no pueden confirmar escrituras. Solo tolera **fallas crash**, no nodos que mienten o se comportan arbitrariamente.

Líderes obsoletos y fencing tokens: un nodo puede pausarse sin estar muerto (p. ej. por garbage collection) y despertar creyéndose líder cuando el cluster ya está en un term superior. Un **fencing token** es un número creciente que permite a un recurso externo rechazar operaciones de una autoridad vieja.

Caso real: Kubernetes + etcd. Kubernetes guarda el estado del cluster (qué apps deben correr, cuántas réplicas, configuraciones) en etcd, que usa Raft. Si etcd pierde la mayoría no puede confirmar cambios del control plane, aunque otros componentes sigan funcionando.

Paxos

Algoritmo clásico de consenso por propuestas y aceptaciones, pensado para que propuestas concurrentes no terminen en decisiones incompatibles.

Roles: Proposer (propone), Acceptor (promete y acepta), Learner (se entera del valor elegido).

1. **Prepare / Promise:** el proposer elige un número único n y envía PREPARE(n). Cada acceptor, si no prometió un número mayor, promete no aceptar propuestas menores a n y responde PROMISE informando si ya había aceptado algún valor y con qué número.
2. **Accept:** con promesas de una mayoría, el proposer envía ACCEPT(n, v); si ningún acceptor había aceptado antes, v puede ser su propio valor (si no, debe respetar el aceptado previamente). Si una mayoría acepta, el valor queda elegido.

Los quórums se intersectan: así una ronda nueva no puede ignorar una decisión anterior. Multi-Paxos repite esto por posición del log. Raft y Paxos persiguen lo mismo: impedir decisiones incompatibles para la misma posición lógica.

Clase 05 — Serializabilidad, anomalías y commit distribuido

Pregunta central: el consenso da un orden total de operaciones, pero falta la corrección lógica de operaciones compuestas: ¿cómo ejecutar operaciones concurrentes sin violar invariantes?

Transacción: unidad lógica de trabajo que se ejecuta de forma atómica y aislada. **ACID:** Atomicidad (todo o nada), Consistencia (preserva invariantes), Aislamiento (como si corriera sola), Durabilidad (persiste tras el commit).

Serializabilidad: una ejecución concurrente es correcta si equivale a alguna ejecución secuencial. Se verifica con el **grafo de precedencia** (nodos = transacciones, aristas = dependencias por conflicto): si tiene ciclo, no es serializable. Es el estándar de corrección, pero exige coordinación fuerte.

Anomalías

Anomalía	Qué pasa	Ejemplo
Dirty read	Se lee un valor no confirmado	T1 escribe, T2 lee, T1 aborta
Non-repeatable read	La misma transacción lee dos veces un dato y obtiene valores distintos	T1 lee, T2 modifica y confirma, T1 relee
Phantom	Una consulta repetida devuelve otro conjunto de filas	T1 lee un conjunto, T2 inserta, T1 relee y aparece un “fantasma”
Write skew	Dos transacciones leen el mismo estado, escriben filas distintas sin conflicto directo y rompen un invariante entre filas	Dos médicos ven que hay otro de guardia, ambos se retiran: no queda ninguno

Niveles de aislamiento (SQL estándar, de menor a mayor)

Nivel	Evita	Todavía permite
Read Uncommitted	—	Dirty reads
Read Committed	Dirty reads	Non-repeatable reads (cada sentencia puede ver una vista nueva)

Nivel	Evita	Todavía permite
Repeatable Read	Non-repeatable reads	Phantoms en la definición estándar (algunos motores dan más)
Serializable	Todas	— (puede hacer esperar o abortar y reintentar transacciones)
Snapshot Isolation	Dirty y non-repeatable reads (foto consistente al inicio)	**Write skew**: no es serializable

Commit distribuido

Problema: un commit que abarca varios nodos debe ser atómico.

- **2PC (two-phase commit):** un coordinador habla con todos los participantes. Fase 1, *prepare*: "¿podés commitear?"; un YES significa "reservé lo necesario y puedo confirmar cuando me digas". Fase 2, *commit*: si todos votan YES, el coordinador registra COMMIT; si alguno dice NO, abort. **Problema:** si el coordinador cae, los participantes quedan bloqueados y nadie puede decidir solo; además requiere muchos mensajes que pueden perderse.
- **3PC:** agrega una fase intermedia *pre-commit* ("estamos listos y entramos en la fase previa a la decisión"). Bajo sus supuestos de sincronía, los participantes usan ese estado y timeouts para avanzar sin depender indefinidamente del coordinador. Reduce el bloqueo, pero **requiere sincronía parcial**.
- **Saga:** secuencia de transacciones locales en distintos servicios; si algo falla, ejecuta **compensaciones** para deshacer los efectos. Desarrollo completo en la Clase 09.

Clase 06 — Storage distribuido: durabilidad, concurrencia y rendimiento

Idea central: el almacenamiento es la capa donde las promesas lógicas se convierten en bytes, esperas, recuperación y riesgos medibles. **Ningún mecanismo elimina trabajo: cada uno decide cuándo, dónde y cómo se paga.**

El storage distribuido conserva decisiones tras un reinicio, coordina lecturas y escrituras simultáneas, encuentra datos con latencia predecible y absorbe carga sin acumular trabajo sin límite. Tiene dos capas: la **capa distribuida** ubica, particiona y replica datos y decide qué nodo tiene autoridad para confirmar; el **motor local** (motor de almacenamiento) traduce eso en bytes persistentes, versiones, índices y recuperación.

Caso CampusShop: una estudiante compra la última unidad (operación con ID estable 9F2A), la API responde “confirmada” y un segundo después el nodo pierde energía. Al reiniciar, la compra debe existir exactamente una vez, el stock no debe reaparecer, un reporte largo debe ver una vista coherente, la consulta de las últimas 20 compras no debe recorrer toda la tabla y la carga sostenida no debe crear colas sin límite.

Las cinco responsabilidades

Responsabilidad	Pregunta	Mecanismo
Registrar decisiones	¿Cómo sobrevive una compra confirmada?	WAL y recuperación
Controlar concurrencia	¿Qué ve un reporte mientras cambian los datos?	MVCC e instantáneas
Elegir ruta física	¿Cómo encontrar 20 filas entre un millón?	Árbol B, planificador, EXPLAIN
Optimizar escrituras	¿Cómo agrupar escrituras y evitar accesos dispersos?	Árbol B o LSM-tree
Proteger capacidad	¿Qué pasa si la compactación no da abasto?	Backlog, regulación, write stall

Estados de una escritura y durabilidad

Una escritura pasa por estados distintos que no hay que confundir: **aceptada** (alguna capa la recibió), **escrita** (entregada a otra capa, quizás en memoria volátil), **durable** (existe evidencia que sobrevive a la falla considerada), **aplicada** (la estructura principal la refleja), **replicada** (otra copia la recibió, no necesariamente durable) y **visible** (una lectura puede verla según el aislamiento).

Frontera de durabilidad: punto desde el cual hay evidencia suficiente para reconstruir la operación tras la falla considerada. Siempre es “durable frente a qué falla”: sobrevivir a un reinicio no es sobrevivir a la destrucción del disco. Dos métricas distintas: **RPO** (máxima pérdida de datos aceptable) y **RTO** (tiempo objetivo para recuperar el servicio).

Camino crítico de una escritura: pasos que deben completarse antes de confirmar (request → WAL durable → actualizar memoria → confirmar); el paso más lento fija la latencia. La E/S **secuencial** (agregar al final de un log) suele ser más barata que la **aleatoria** (ubicaciones no contiguas), por eso conviene registrar primero en un log y actualizar páginas después.

Vocabulario clave: *commit* = decisión de aceptar los efectos (puede tomarse antes de escribir las páginas); *ACK* = respuesta al cliente cuando se cumplió la garantía; *write/append* = copiar bytes a una capa (pueden seguir en caché); *flush* = mover datos a una capa inferior (no implica durabilidad); *fsync* = pedir al SO que sincronice con el medio persistente; *página sucia* = página modificada en memoria aún no escrita (no es un error, se retrasa a propósito). Ante "ya está escrito", preguntar: ¿en qué capa terminó y qué falla todavía puede borrarlo?

WAL (Write-Ahead Logging)

Registro secuencial de modificaciones y decisiones transaccionales. Resuelve el problema de caerse a mitad de escribir varias páginas (compra escrita pero stock sin descontar) y evita escribir todas las páginas antes de cada ACK. **El WAL conserva evidencia; las páginas materializan el estado.** No elimina trabajo: cambia el orden del riesgo y del trabajo físico.

Las dos reglas:

1. **Log antes que página:** registro WAL → flush durable → página. Evita una página persistida sin evidencia del cambio que la produjo.
2. **COMMIT antes que ACK:** registro COMMIT → flush durable → ACK. Evita decir "confirmada" y perder la decisión.

Secuencia completa: solicitud con ID estable → cambios preparados en memoria → registros en el WAL (todavía volátiles) → páginas sucias (memoria con stock 0, disco con 1) → registro COMMIT (volátil) → **flush durable del WAL** → **recién ahí ACK**, sin esperar las páginas → si hay crash se pierde RAM pero queda el WAL → recovery hace **REDO** y el disco queda en 0.

Momento del crash	Evidencia durable	Resultado correcto
Antes del COMMIT durable	Quizás cambios, sin COMMIT	La compra no aparece; lo incompleto se descarta o revierte
Después del COMMIT, antes de las páginas	Cambios + COMMIT en el WAL	REDO reconstruye compra, stock e índice
Después del COMMIT y de las páginas	WAL y páginas	Se verifica; repetir REDO no debe duplicar efectos

- **REDO:** reaplica cambios confirmados no materializados. Debe ser repetible sin duplicar (se usan los LSN de las páginas).
- **UNDO:** revierte efectos de transacciones no confirmadas que llegaron al almacenamiento.

- **LSN (log sequence number):** posición en el WAL, no un reloj. *Insert LSN* (hasta dónde se insertó en buffers), *write LSN* (hasta dónde se entregó al SO), *flush LSN* (hasta dónde es durable).
- **Group commit:** un acceso físico amortiza varios COMMIT; cada transacción puede esperar un poco, pero ningún ACK sale antes de que su COMMIT sea durable.
- **Checkpoint:** punto durable que acota desde dónde empieza la recuperación. Más frecuente: menos WAL a reproducir pero más E/S y picos. Más espaciado: operación más suave pero recuperación más larga.

Lo que el WAL no garantiza: exactly-once. Si el cliente reintenta, hace falta reconocer que es la misma operación: clave idempotente estable (9F2A), restricción de unicidad, persistir juntos ID, efecto y resultado, y devolver el resultado anterior. WAL, checkpoints, backups, replicación e idempotencia son mecanismos distintos y no se reemplazan entre sí.

MVCC (Multi-Version Concurrency Control)

WAL responde cómo sobrevive una transacción a un crash; MVCC responde cómo leer mientras otros escriben sin lecturas incoherentes. Mantiene **varias versiones** de cada fila y aplica **reglas de visibilidad**: una escritura crea una versión nueva sin borrar la anterior.

- **Snapshot:** instantánea **lógica** que define qué versiones ve una lectura. No copia la base. En Read Committed cada sentencia puede tomar un snapshot nuevo; en Snapshot Isolation se mantiene durante toda la transacción.
- **Modelo simplificado de versión:** (x, valor, t_inicio, t_fin).
- **Ejemplo:** el reporte TR fija su snapshot y lee v1 = 120. TW crea v2 = 135 y confirma. TR vuelve a leer y sigue viendo 120; un lector nuevo ve 135. Cuando TR termina y nadie necesita v1, su espacio se recupera.
- **Read Committed vs. SI:** T1 lee $1.000, T2 cambia a $500 y confirma; en la segunda lectura T1 ve $500 en RC y $1.000 en SI.

Costos: espacio extra, índices con referencias a versiones muertas, páginas menos densas, más E/S, presión sobre la caché y garbage collection. **VACUUM** recupera espacio reutilizable de versiones que ya nadie necesita (normalmente no achica el archivo).

Condición de carrera y write skew bajo MVCC: dos transacciones leen la misma instantánea, actualizan filas distintas y ambas confirman, violando un invariante de varias filas. Soluciones: aislamiento serializable, lock explícito, validación de dependencias al confirmar, reformular el invariante para que choquen sobre un recurso común u operación atómica especializada. **MVCC administra versiones; no da serializabilidad por sí solo.**

Índices, planificador y EXPLAIN

MVCC decide qué versión ver; el índice ayuda a encontrarla sin un sequential scan. Un índice no es gratis: se mantiene en cada escritura.

- **Árbol B:** índice ordenado y balanceado (todas las hojas a la misma profundidad) cuyos nodos son páginas. Páginas internas con separadores, hojas con claves y referencias a filas. Por su alto *fan-out* llega a millones de filas con pocos niveles. Ideal para búsquedas puntuales y rangos; cuesta escritura en varias páginas y divisiones cuando se llenan.
- **Índice compuesto:** usa más de una columna.
- **Planificador (query planner):** compara rutas posibles con estadísticas y un modelo de costo estimado. Que exista un índice no obliga a usarlo.
- **EXPLAIN** muestra el plan estimado sin ejecutar; **EXPLAIN ANALYZE** ejecuta y agrega tiempos y filas reales. Los planes se leen de abajo hacia arriba. No se agrega un índice porque “parece útil”: hipótesis, plan y medición.

LSM-tree (Log-Structured Merge-tree)

Acepta muchas escrituras sin actualizar páginas dispersas: “throughput ahora, deuda después”.

Componentes: WAL (durabilidad), **MemTable** (estructura ordenada en RAM que recibe escrituras), **immutable memtable** (memtable congelada al llegar al umbral; se sigue leyendo mientras se crea una nueva activa), **SSTable** (archivo ordenado por clave e **inmutable**), **compactación** y **manifest** (metadatos de qué SSTables y niveles forman el estado actual; sin él no se sabe qué archivos usar tras recuperar).

Escritura: append al WAL → actualizar la memtable → ACK (sin esperar el flush) → al llenarse, **flush** secuencial a una SSTable en L0 → más tarde, **compactación**.

Lectura: memtable activa → immutable memtables → SSTables de L0 → niveles compactados. Una clave puede estar en varios archivos. L0 es especial porque sus SSTables vienen de flushes independientes y sus rangos se solapan. Para reducir el costo: **Bloom filters** (responden “no está” con certeza o “tal vez está”; nunca garantizan presencia), índices por archivo y bloque, cachés y compactación.

Compactación: fusión ordenada (merge) de SSTables que además selecciona versiones vigentes, conserva las históricas necesarias, gestiona **tombstones** (marcas de borrado que se guardan hasta que las versiones anteriores no puedan resurgir) y reduce solapamiento. Flush = “datos nuevos de RAM a disco”; compaction = “reorganizar muchos archivos en disco”. Consume disco, CPU y espacio temporal y puede subir el p99.

Cadena de saturación: ingreso mayor que la capacidad de compactar → **backlog** creciente → más SSTables en L0 → lecturas más caras → **regulación de admisión** (retrasar, bloquear o limitar tokens/bytes) → en el caso extremo **write stall** (las escrituras no avanzan hasta reducir el backlog; algunos sistemas responden sobrecarga). La pausa protege una cola finita pero **no crea capacidad**: si es frecuente, revisar carga, hardware y política de compactación.

Amplificaciones

"Amplificación" no es error: es trabajo físico mayor que la operación lógica.

- **Write amplification** = bytes físicos escritos / bytes lógicos ingresados. Más desgaste de SSD, más recursos y competencia con escrituras nuevas.
- **Read amplification:** encontrar una clave obliga a consultar varias estructuras.
- **Space amplification** = espacio físico ocupado / espacio lógico vivo.

Estrategia de compactación	Cómo organiza	Favorece	Costo
Leveled	Niveles con rangos poco solapados	Lecturas puntuales y control de espacio	Más write amplification (los datos se reescriben al bajar de nivel)
Tiered / Universal	Permite varios runs solapados	Menos reescritura	Más archivos que leer y más espacio

Árbol B vs. LSM: el árbol B paga en el camino crítico (varias páginas por escritura) y lee con un recorrido corto; el LSM evita accesos dispersos antes del ACK y paga después en flush, compactación, lecturas y espacio. Es una orientación, no una ley: se decide por patrón de acceso, garantías, capacidad sostenible, p95/p99 y amplificaciones medidas, no por el nombre del producto.

Método reusable para decidir una arquitectura

1. **Describir la carga:** proporción lectura/escritura, tamaños, volumen y crecimiento, concurrencia y **skew** (trabajo no uniforme: un producto popular genera un hotspot).
2. **Declarar garantías:** durabilidad, RPO, RTO, consistencia, frescura.
3. **Elegir una hipótesis de mecanismo.**
4. **Identificar el costo desplazado** y cómo se lo controla.
5. **Definir métricas** observables.
6. **Diseñar un experimento capaz de refutar la hipótesis:** la elección se defiende mejor cuando se sabe qué resultado obligaría a reconsiderarla.

Clase 07 — Logs distribuidos y procesamiento de eventos

Idea central: usar una historia durable como interfaz entre productores y consumidores que pueden fallar, reiniciarse y avanzar a distinto ritmo, **sin confundir publicación, entrega, progreso y efecto de negocio**.

Conceptos base

- **Log:** secuencia ordenada, append-only, de registros que se conservan y se pueden releer. Guarda la historia de hechos, no solo el estado final.
- **Comando vs. evento vs. registro:** el comando es una intención que puede aceptarse o rechazarse ("confirmá el pedido"); el evento es un hecho ya ocurrido (OrderConfirmed(9F2A)); el registro es su representación almacenada con identidad y metadatos.
- **WAL vs. log distribuido:** el WAL es interno al motor y protege la durabilidad de una base; el log distribuido comparte un historial ordenado entre servicios.
- **Actores:** productor (publica), broker (servidor que almacena y sirve particiones), consumidor (lee y produce un efecto). Cluster = conjunto de brokers.
- **Apache Kafka:** plataforma de event streaming donde productores publican, brokers guardan logs particionados y consumidores leen manteniendo su propio progreso.
- **Log vs. cola de mensajes:** en una cola el mensaje se consume y desaparece; en el log los registros persisten, varios consumidores leen la misma historia cada uno con su posición y se puede hacer **replay** (releer registros ya procesados). Releer no debe duplicar efectos de negocio.
- **Nodo, proceso y hardware:** nodo = máquina física o virtual; proceso = programa en ejecución; hardware = CPU, RAM (volátil), NIC (red) y SSD (persistente si la escritura fue durable).

Por qué no fan-out síncrono: si Checkout llamara y esperara a QR, Notificaciones y Analítica, el componente más lento fijaría la latencia y una dependencia secundaria haría fallar una compra válida. Antes del OK solo debe ocurrir lo que define la validez (reservar stock, autorizar pago, guardar pedido + intención de publicar); el resto es asíncrono y sale del camino crítico.

Beneficios y su desafío asociado: Checkout no espera (garantizar publicación durable), varias apps leen la misma historia (administrar el progreso de cada una), recuperación por replay (evitar efectos duplicados), reparto en particiones (preservar el orden necesario), alto volumen (controlar lag y sobrecarga), vistas derivadas (manejar eventos atrasados).

Topics, particiones, offsets y keys

- **Topic:** categoría lógica de registros (ej. orders.lifecycle) dividida en **particiones**, cada una un log append-only ordenado. **El orden existe solo dentro de una partición**; no hay orden global gratuito.
- **Offset:** posición de un registro en su partición (partición + offset lo ubica).
- **Key de particionamiento:** el productor elige una key estable y una función determinista decide la partición: partición = h(key) mod P. Todos los eventos de un pedido caen en el mismo log y conservan su orden, mientras pedidos distintos avanzan en paralelo. La key **no crea** el orden causal: si se publica desordenado, Kafka no lo corrige.
- **aggregate_seq:** posición del evento en la historia de su entidad; permite detectar huecos, repeticiones o desorden. No confundir con el offset (posición en el log).

Anatomía de un registro (las identidades que no hay que mezclar): event_id (el evento concreto), aggregate_id (la entidad, ej. 9F2A), aggregate_seq (posición en la historia de la entidad), type (qué hecho ocurrió) y partición/offset (dónde está guardado).

Particionar gana orden por entidad, paralelismo entre entidades, distribución de escritura y consumo y recuperación por partición; renuncia al orden global, al paralelismo dentro de una misma key y a rebalancear una key sin mover su historia. La partición es una frontera de orden, capacidad y paralelismo.

Buena key: agrupa exactamente lo que necesita ordenarse junto (causalidad), es estable, tiene cardinalidad suficiente, granularidad mínima, distribución pareja y se puede observar (bytes, QPS y lag por key). No hay key universal: depende del invariante del topic.

Hot keys y hot partitions

Una **hot key** concentra tráfico desproporcionado y genera una **hot partition**, cuello de botella con lag creciente. Agregar consumidores no ayuda: paralelismo activo ⩽ min(P, C). Opciones: rediseñar la key, dividir la entidad, aceptar el cuello de botella si el orden estricto importa más, o **agregación en dos etapas** (cada partición calcula un parcial y una segunda etapa los combina; sirve para conteos, sumas, mín/máx e histogramas, no para partir la historia ordenada de una entidad).

Replicación y publicación

- Cada partición tiene una **réplica líder** (recibe publicaciones y fija el orden) y **seguidoras** que copian su log; si el líder cae, una réplica elegible asume.
- **Factor de replicación (RF):** cuántas copias se configuran (RF = 3: un líder y dos seguidoras). No dice cuántas deben confirmar antes de responder.

- **ISR (in-sync replicas):** réplicas suficientemente sincronizadas que pueden participar en la garantía de confirmación.
- **ACK del productor:** confirmación de que se cumplió cierta condición de publicación. Un timeout no dice si el evento quedó en el log; reintentar ingenuamente puede duplicarlo.
- **Productor idempotente:** combina identidad, época y números de secuencia para que el broker reconozca reenvíos del mismo lote. No impide que dos productores distintos publiquen el mismo hecho lógico.

Consumidores: progreso, fallas y garantías

- **Poll:** pedir un lote de registros (polling = repetirlo). **Position:** próximo offset que la instancia intentará leer, quizás solo en memoria. **Committed offset:** progreso durable desde el que el grupo reinicia tras una falla; indica lectura confirmada, no que el efecto se haya aplicado.
- **Consumer group:** instancias de la misma app con un group.id; el broker reparte particiones completas y cada partición tiene una sola instancia activa (las sobrantes quedan ociosas).
- **Rebalanceo:** redistribución de particiones cuando una instancia cae, entra o sale, o cambian las particiones. Cuesta pausas, revocación y reasignación, trabajo en curso perdido, restauración de estado y cachés frías.
- **Consumer lag:** trabajo pendiente entre el final del log y el progreso confirmado; se mide en offsets o en antigüedad del evento más viejo sin procesar. Es un síntoma (producción y consumo a distinto ritmo) con varias causas posibles: hot partition, restore, dependencia lenta.

Estrategia	Orden	Riesgo ante crash
At-most-once	Commit del offset antes del efecto	Se pierde el efecto (el QR nunca se emite)
At-least-once	Efecto antes del commit	Replay: el efecto puede duplicarse si no es idempotente
At-least-once + idempotencia	Efecto + registro en inbox atómicos	Un solo efecto lógico

Inbox idempotente: guardar el event_id procesado junto con el efecto en una misma transacción; si el registro reaparece, la inbox prueba que ya se aplicó. Idempotencia no es leer una sola vez: es que releer no cambie el resultado.

Exactly-once solo vale dentro de una frontera coordinada: hay que preguntar exactamente una vez **qué**, **en qué sistema** y **frente a qué fallas**.

Dual write y transactional outbox

Dual write: una operación modifica dos sistemas sin transacción común (base de pedidos y broker). Base primero y crash: la compra existe pero el evento falta. Broker primero y rollback: los consumidores ven una compra que no existe.

Transactional outbox: en el mismo COMMIT local que confirma el pedido se inserta una fila en una tabla outbox de la misma base. Si hay commit existen ambos; si aborta, ninguno. No significa que el evento ya esté en Kafka, sino que hay una **intención durable de publicarlo**.

Un **relay** toma las filas pendientes y las publica: por **polling** (consultar la tabla periódicamente) o por **CDC** (change data capture, leyendo cambios confirmados del WAL/binlog; Debezium es una implementación). El relay puede publicar y perder el ACK, así que la outbox **no evita duplicados**: los consumidores downstream necesitan identidad e idempotencia.

	Outbox	Inbox
Lado	Productor	Consumidor
Guarda	Eventos pendientes de publicar	IDs de eventos ya procesados
Resuelve	Estado confirmado sin intención de publicar	Efectos duplicados por replay
Atómico	Cambio de negocio + fila outbox	ID procesado + efecto local

Tres historias distintas: el **WAL** (lo administra el motor, recupera páginas y transacciones), la **outbox** (aplicación + base local, conserva la intención de publicar) y el **log distribuido** (cluster de brokers, comparte registros entre consumidores).

Streaming

- **Stream:** secuencia potencialmente ilimitada (ventas en vivo); un **batch** tiene final conocido (ventas de ayer). **Pipeline:** etapas que filtran, transforman, agrupan o enrutan.
- **Stateless vs. stateful:** stateless procesa cada evento por separado; stateful recuerda información entre eventos (contar ventas). El estado vive en un **state store** (RAM para velocidad, SSD local para persistencia), respaldado por un **changelog** (log de cambios para reconstruirlo en otro nodo) y **checkpoints/snapshots**.

- **Tiempos:** *event time* (cuándo ocurrió en el dominio), *ingestion time* (cuándo llegó al sistema), *processing time* (cuándo se procesa). Para reconstruir un intervalo hace falta event time de una fuente confiable.
- **Ventana temporal:** delimita qué eventos entran en una agregación. Esperar mucho retrasa resultados; cerrar pronto los deja incompletos.
- **Watermark:** señal de progreso del event time para decidir cuándo cerrar ventanas, W_i = max(event_time observado en P_i) − Δ (margen de desorden). No garantiza que no lleguen eventos anteriores. El watermark global lo limita la partición activa más atrasada; una partición inactiva puede marcarse como tal para no bloquearlo.
- **Late data:** se puede corregir, ignorar o desviar a otro flujo.
- **SLO:** meta cuantificable (cuánto atraso se acepta) para dimensionar particiones, consumidores, rebalanceos y política de eventos tardíos. Ver Clase 12.

Clase 08 — Consenso aplicado, CAP, PACELC, Dynamo y Spanner

Pregunta central: ¿qué pasa cuando dos nodos quieren decidir sobre el mismo dato de forma incompatible? Replicar no implica coordinar. La clase repasa causalidad (ver Clase 03) y Raft (ver Clase 04) y agrega lo siguiente.

Invariantes y fallas parciales

- **Invariante:** condición lógica que debe ser siempre verdadera (reservas VIP confirmadas ⩽ 1). Se pueden almacenar ambas solicitudes; lo que no se puede es confirmar ambas. **La consistencia necesaria depende de los invariantes a proteger.**
- **Timeout ≠ servidor caído.** Puede significar que el mensaje no llegó, que el servidor cayó antes de procesar, que procesó y se perdió la respuesta, o que está lento.
- **Estados de una réplica:** sana y comunicada; **sana pero aislada** (la más peligrosa: puede atender con información vieja); caída o pausada.

Problema	Mecanismo
No saber qué evento ocurrió antes	Causalidad y relojes lógicos
Decisiones incompatibles entre nodos	Consenso (Raft, Paxos)
La red se divide	CAP
Consistencia fuerte vs. latencia	PACELC
Seguir aceptando operaciones durante fallas	Dynamo
Transacciones globales que respeten el tiempo real	Spanner y TrueTime

CAP

Durante una partición, N4 y N5 (minoría) no distinguen “Sofía ya compró y no nos enteramos” de “nadie compró”. Si confirman a Tomás pueden vender dos veces; si rechazan, Tomás no compra.

- **C (consistencia fuerte):** se comporta como una única copia autoritativa.
- **A (disponibilidad):** toda solicitud a un nodo no fallado termina con una respuesta válida de la operación.
- **P (tolerancia a particiones):** la red puede dividirse en grupos de nodos vivos incomunicados.

Con partición no se pueden tener C y A a la vez. Como las particiones ocurren, hay que elegir:

Opción	Comportamiento	Gana	Pierde
CP	La minoría espera o rechaza (como Raft sin mayoría)	Preserva el invariante	Disponibilidad de la operación en la minoría
AP	Ambos lados aceptan con información local	Ambos lados siguen operando	Consistencia; hay que reconciliar después, si se puede

Uptime ≠ disponibilidad CAP: uptime es el porcentaje de tiempo levantado; disponibilidad CAP exige respuesta válida de la operación; disponibilidad semántica es la proporción de operaciones que cumplen la promesa del negocio. Un 503 rápido prueba que el proceso vive, no que la compra esté disponible.

PACELC

Si hay **P**artición, el trade-off es **A** vs. **C**; si no (**E**lse), es **L**atencia vs. **C**onsistencia. Coordinar cuesta incluso con la red sana.

Modelos de consistencia (comparación fina)

- **Recencia:** qué tan nuevo es el estado observado (medible como atraso).
- **Linearizabilidad:** cada operación parece atómica entre invocación y respuesta y respeta el orden real de las no superpuestas. Tomás no puede ver AVAILABLE si la compra de Sofía ya fue confirmada.
- **Serializabilidad:** el resultado de transacciones equivale a algún orden serial, **sin exigir el orden real**.

- **Consistencia externa:** serializabilidad + respeto del orden temporal observado por los clientes.

Quórums: R + W > N no es consenso

N = réplicas preferidas de una clave, W = respuestas para aceptar una escritura, R = respuestas para una lectura. R + W > N garantiza intersección entre conjuntos ideales, pero **no** garantiza linearizabilidad, un líder, orden total, una versión autoritativa ni resolución semántica de conflictos. Intersección no es consenso.

Contraejemplo, sloppy quorum: si para seguir disponible la escritura va a N1, N6 y N7 (N6 y N7 fuera del conjunto preferido) y la lectura consulta N3, N4 y N5, no hay intersección real. Para una entrada única, quórums tipo Dynamo pueden detectar el conflicto pero no fusionar dos reservas sin violar el invariante: no reemplazan al consenso.

Dynamo

Almacén clave–valor de Amazon que prioriza disponibilidad: acepta operaciones en varias réplicas, conserva versiones concurrentes, detecta divergencias y repara después. La disponibilidad se paga con versiones, reconciliación, hints y trabajo de reparación.

1. **Localizar réplicas:** hash de la key → *preference list* (no todos los nodos guardan todas las claves).
2. **Escribir hasta W respuestas:** cumple el requisito configurado, no es una decisión global por consenso.
3. **Conservar versiones concurrentes** en lugar de descartar una.
4. **Responder según la política de disponibilidad**, incluso usando réplicas alternativas (sloppy quorum).
5. **Reparar** con hinted handoff, read repair y anti-entropy.

- **Sloppy quorum + hinted handoff:** si una réplica preferida falla, la copia se guarda temporalmente en otro nodo junto con un **hint** (a quién pertenece); al volver el dueño, se le entrega. Cuesta almacenamiento, red, E/S y reparación.
- **Read repair:** corrige diferencias detectadas al leer; no cubre datos que nadie lee.
- **Anti-entropy:** compara réplicas periódicamente sin depender de lecturas.
- **Conflictos fusionables vs. no fusionables:** un carrito se puede fusionar en una versión descendiente; la última entrada VIP no. Resolver depende de la semántica, no solo del almacenamiento.
- **Convergencia:** requiere que la capacidad de reparación supere la tasa de divergencia durante la recuperación. “Eventual” no es magia: necesita mecanismos y recursos.

Spanner y TrueTime

Base de Google distribuida globalmente con transacciones entre grupos de datos y orden compatible con lo que ven los clientes. Problema: una compra toca inventario, pagos y derechos del usuario en grupos distintos, y debe ser atómica (que Sofía no pague sin recibir su entrada).

- **Consenso localizado:** cada grupo de datos tiene su propia replicación y consenso; no hay un líder global. Si una transacción toca varios grupos, un coordinador los organiza para que confirmen juntos con un timestamp de commit común.
- **TrueTime:** API que en lugar de una hora exacta devuelve un intervalo [earliest, latest] que contiene el tiempo real. No elimina la incertidumbre: la hace **explícita y acotada**.
- **Commit:** (1) el coordinador elige un timestamp s que respeta los timestamps de preparación, la cota superior de TrueTime y las dependencias; (2) no responde de inmediato porque el tiempo real podría estar en cualquier punto del intervalo; (3) **commit wait**: espera hasta que earliest > s, es decir, hasta poder probar que s ya pasó. Costo: latencia adicional.
- **Consistencia externa:** como T1 solo se confirma cuando s_1 está en el pasado, cualquier T2 que empiece después queda ordenada detrás. Se llama “externa” porque el orden debe coincidir con lo observado desde afuera. **Para el parcial: consistencia externa = serializabilidad + orden temporal observado por los clientes.**

Clase 09 — Fronteras de arquitectura: monolito modular, servicios, eventos y sagas

Caso conductor: compra de EspadaSolar#17 en el juego Asteria. No puede romperse: la pieza tiene a lo sumo un propietario, no se cobra dos veces el mismo intento, una compra confirmada no desaparece. Puede demorarse: ranking, notificación, analítica, historial derivado.

Idea central: mover una frontera no elimina la regla de negocio; transforma una transacción local en un protocolo distribuido que debe sobrevivir fallas parciales. Primero se identifica la responsabilidad y su invariante; recién después se decide si es módulo, proceso o servicio.

Conceptos fundamentales

- **Dominio:** parte de la realidad con reglas y vocabulario propios (Mercado, Inventario, Pagos).
- **Capacidad:** responsabilidad estable del negocio que agrupa reglas, datos y operaciones (gestionar la propiedad: reservar, transferir, liberar, consultar). No es un endpoint.

- **Invariante:** regla verdadera en todo estado válido.
- **Frontera:** límite explícito que define qué responsabilidades y datos quedan de cada lado y por qué puertas se interactúa.
- **Autoridad:** quién tiene la última palabra sobre un hecho (Inventario decide el propietario). **Responsabilidad (ownership):** qué equipo mantiene, opera y responde por esa parte.
- **Dato autoritativo:** la verdad oficial para decidir (propietario(#17) = Sofía); Ranking solo tiene una vista derivada.
- **Unidad de publicación, puesta en producción y rollback de versión.** Volver a una versión anterior **no deshace** pagos, eventos ni transferencias confirmadas: eso requiere reparación de negocio.
- **Autonomía:** grado en que una parte puede cambiar, desplegarse, escalar o recuperarse sin arrastrar al resto. Tipos: de cambio, de despliegue, de escalado y de recuperación. Separar procesos no la crea por sí solo.

Monolito modular vs. microservicios

	Monolito modular	Microservicios
Despliegue	Una sola unidad	Cada servicio por separado
Comunicación	Llamadas locales	Contratos de red o eventos
Invariantes	Transacciones locales	Protocolos distribuidos
Ventajas	Simple de operar, fácil mover fronteras internas	Escalado, despliegue y aislamiento de fallas independientes
Costo	Escala y despliega todo junto	Latencia, timeouts, mensajes perdidos, fallas parciales

Vocabulario: **módulo** (frontera de código), **proceso** (programa en ejecución con memoria y fallas propias), **servicio** (capacidad accesible por contrato, normalmente de red), **microservicio** (servicio con capacidad coherente y autonomía razonable).

Motivos para separar: necesidades de escalado distintas (Ranking en torneos), ritmos de cambio distintos, aislamiento de fallas (que Chat no tumbe al Mercado), controles de seguridad específicos (Pagos). **Antes, probar alternativas más simples:** replicar y balancear, caché, diferir cálculos y limitar recursos, reservas condicionales atómicas.

Monolito distribuido (antipatrón): dividido físicamente pero acoplado (base compartida, versiones coordinadas). Paga los costos de distribuir sin obtener autonomía.

Qué separar primero: Ranking (picos propios, tolera atraso, su falla no frena compras). Inventario es más riesgoso: protege el invariante de propiedad y está en el camino crítico.

Bounded contexts y autoridad de datos

- **Bounded context:** frontera dentro de la cual vocabulario, modelo y reglas tienen significado coherente ("espada" no significa lo mismo en Mercado que en Inventario). Los contextos se comunican por IDs, comandos, consultas y eventos, sin objetos mutables compartidos. **No es necesariamente un microservicio:** pueden ser módulos de un monolito.
- **Frontera semántica** (separa modelos y responsabilidades) vs. **frontera física** (procesos que se comunican por red). Se puede tener buena arquitectura sin distribuir todo.
- **Ownership de datos:** solo la autoridad modifica su dato; los demás consultan o consumen eventos. Ser dueño (modificar y preservar reglas) no es lo mismo que tener una copia (consultar o derivar).
- **Reserva condicional atómica:** `UPDATE items SET estado='RESERVADA', reservado_por='Sofía' WHERE id=17 AND estado='DISPONIBLE'`, exitosa solo si actualizó exactamente una fila. Protege localmente el invariante ante solicitudes concurrentes.

Cuando el invariante cruza fronteras (reservar, autorizar pago, transferir, confirmar) no hay un ROLLBACK común: hace falta un **protocolo distribuido** que defina ID del intento, quién decide cada paso, qué estado se guarda, qué significa un timeout, cómo se reintenta y qué hacer si un paso falla.

Comunicación entre servicios

- **RPC** se parece a una llamada local pero atraviesa la red, serializa, puede perder request o respuesta, no comparte transacción y puede dar resultados ambiguos.
- **Timeout ambiguo:** es una observación local, no prueba de falla. Mercado podría creer que Inventario no reservó cuando sí lo hizo; con Pagos, cobrar dos veces.
- **Idempotencia con attempt_id:** Mercado genera un ID por intento (compra_123); Inventario guarda **durablemente** la reserva y su resultado asociado; ante un reintento devuelve el resultado registrado. Mismo intento = mismo ID; nueva operación = ID nuevo. Si los IDs solo vivieran en memoria, se perderían en un reinicio.
- **Cadenas síncronas:** las latencias se acumulan y la disponibilidad del flujo depende de cada paso obligatorio. Más servicios no implica más disponibilidad.
- **Ruta crítica:** solo lo necesario para responder (validar oferta, reservar, autorizar pago, transferir, confirmar). Ranking, notificaciones y analítica van después, de forma asíncrona.

- **Comando** (pide a una autoridad que intente algo; puede rechazarse), **evento** (afirma algo ya confirmado), **consulta** (pide información sin modificar). RPC es un mecanismo; comando, evento y consulta son semántica. Que un mensaje pase por un broker no lo vuelve evento.

Eventos y outbox (repaso aplicado)

Un **contrato de evento** define identidad (event_id, purchase_id, versión), semántica exacta (hecho confirmado, no intención), esquema, evolución (agregar o retirar campos sin romper lectores viejos) y entrega/orden (duplicados, orden, replay posibles). El dual write y la outbox funcionan como en la Clase 07: el relay lee PENDING, publica, recibe confirmación, marca publicado y repite. La entrega es **at-least-once**, así que el consumidor (Ranking) usa una inbox para no sumar puntos dos veces. La outbox registra atómicamente cambio local + evento pendiente, pero **no** hace que Inventario, Mercado y Pagos compartan un COMMIT.

Métricas del atraso: filas pendientes en outbox, edad del evento pendiente más antiguo, reintentos, tiempo extremo a extremo y atraso y tiempo de reconstrucción de Ranking. No basta con ser recuperable en teoría: hay que poder ponerse al día.

Sagas

Saga: proceso de negocio durable que coordina una operación mediante varias transacciones locales y guarda dónde quedó para continuar, consultar, reintentar o compensar. Persiste saga_id e ID del intento, estado actual, deadline, comandos y respuestas, y último resultado durable. No es una serie de llamadas: debe sobrevivir a fallas entre pasos.

- **Resultado incierto:** si se pierde la respuesta de AuthorizePayment(456), la saga no asume falla ni cobra de nuevo: registra **PAYMENT_UNKNOWN** y resuelve consultando por ID estable o reintentando de forma idempotente.
- **Compensación:** acción de negocio nueva que neutraliza otra ya confirmada (anular autorización, liberar reserva, reembolsar). **No es rollback:** la historia registra ambas acciones, y la compensación también puede fallar, expirar o duplicarse, así que necesita idempotencia y estado durable.
- **Saga vs. transacción distribuida:** la saga no vuelve atómico lo ya confirmado; coordina los pasos siguientes y las reparaciones, con estados intermedios esperables (espada reservada con pago pendiente) que hay que controlar.

	Coreografía	Orquestación
Cómo funciona	Cada servicio reacciona a eventos y decide su paso	Un coordinador guarda el estado durable y decide el próximo paso
Ventaja	Bajo acoplamiento; fácil sumar consumidores	Estado global en un lugar; más fácil manejar timeouts, reintentos, compensaciones, deadlines y revisión manual
Desventaja	Difícil ver el estado global (¿quién sabe que el pago quedó incierto? ¿quién compensa?)	Puede concentrar demasiada lógica y es otro componente a operar; no reemplaza la autoridad de los servicios

Migración y operación

- **Strangler Fig:** migración incremental: (1) coexisten viejo y nuevo, (2) se comparan resultados, latencia y costo, (3) se desvía tráfico gradualmente, (4) se retira lo viejo con evidencia. En Ranking: alimentar ambos con los mismos eventos, comparar puntajes (considerando atraso), mandar una fracción de lecturas al nuevo y crecer.
- **Canary deployment:** exponer una versión nueva a una pequeña fracción de usuarios observando errores, latencia, exactitud y atraso, con vuelta atrás si empeora. Strangler es el patrón de migración; canary, la técnica de despliegue gradual.
- **Rollback no deshace efectos de negocio:** el sistema viejo debe poder trabajar con los datos generados durante la convivencia.
- **Preparación operativa:** CI (integrar y validar seguido), CD (*continuous delivery*: listo para producción con promoción manual; *continuous deployment*: despliegue automático), observabilidad (logs, métricas, trazas distribuidas), SLI/SLO (ver Clase 12), guardia y **runbooks**. Cada servicio nuevo es una responsabilidad operativa real.
- **¿Valió la pena separar?** Medir tiempo hasta producción, publicaciones sin coordinación, escalado independiente, alcance de incidentes, p99 y errores, y bloqueos entre equipos.

Método para justificar una frontera (aplicado a Ranking):

1. **Problema y evidencia:** los torneos generan picos que suben el p99 de compra.
2. **Regla, dato y autoridad:** Ranking es una proyección; Mercado e Inventario conservan su autoridad.
3. **Alternativa más simple:** caché, procesamiento diferido o límites de recursos en el monolito.

4. **Costo, falla y mecanismo:** separarlo exige eventos confirmados, outbox, idempotencia, manejo de atraso y monitoreo.
5. **Métricas y revisión:** comparar p99 de compra, lag de Ranking, incidentes y tiempo hasta producción antes y después.

Clase 10 — Observabilidad: tracing, métricas, logs y debugging en producción

Idea central: observabilidad distribuida no es mirar dashboards ni abrir tres pestañas; es diseñar el software para que transporte contexto causal y pueda explicarse a sí mismo cuando falla.

Por qué es distinto a un monolito

En un monolito hay stack trace, un solo espacio de ejecución y un reloj local. Al distribuir se pierde el stack trace entre servicios, no hay reloj global confiable (ordenar logs por timestamp puede contar una historia falsa), hay tiempo no observable (la latencia del usuario supera la de cada servicio) y **todos los servicios pueden verse sanos mientras el usuario falla**.

Para qué monitorear: detección temprana, diagnóstico, decisiones con datos, contención del daño (activar circuit breakers) y aprendizaje continuo.

Monitoreo vs. observabilidad

- **Monitoreo** (¿funciona?): indicadores y alertas definidos de antemano (CPU > 90 %, errores > 5 %). Detecta el síntoma, no explica la causa. Cubre los **known-unknowns** (sabemos qué preguntar).
- **Observabilidad** (¿por qué?): inferir el estado interno a partir de las salidas, aun ante problemas no anticipados: los **unknown-unknowns** (“¿por qué fallan solo los pagos de cierto cliente los viernes?”). Si para cada problema nuevo hay que agregar logs y redesplegar, el sistema no era observable.

Los tres pilares

Pilar	Responde	Característica	Herramienta típica
Métricas	¿Qué pasa globalmente?	Numéricas y agregadas; tendencias y alertas	Prometheus

Pilar	Responde	Característica	Herramienta típica
Logs	¿Qué evento ocurrió?	Detalle de eventos concretos; hay que correlacionarlos	Elasticsearch
Trazas	¿Por dónde pasó la solicitud y cuánto tardó cada paso?	Recorrido individual entre servicios	Jaeger

Antipatrón de las tres pestañas: a las 4 a. m. el ingeniero salta entre herramientas aisladas y correlaciona a mano por timestamps, con riesgo de conclusiones falsas. **Solución, trenza única de datos:** un trace_id acompaña la solicitud y aparece en logs, trazas y (vía exemplars) métricas, para navegar métrica → traza → logs. No elimina los pilares: los conecta con contexto compartido.

Contexto

- **Hard context:** identificadores que establecen vínculos causales explícitos, sin adivinar por timestamps: **trace_id** (recorrido completo), **span_id** (operación individual), **parent_span_id** (qué operación la originó).
- **Soft context:** atributos para agrupar y filtrar (tenant_id, region, customer_id, hostname); definen las dimensiones de análisis pero no garantizan causalidad.

El hard context dice qué está relacionado; el soft context ayuda a descubrir qué características explican el problema.

OpenTelemetry

Antes cada pilar tenía su agente (Prometheus por scraping, Filebeat para logs, Jaeger para trazas), con consumo extra de recursos y acoplamiento al proveedor. OpenTelemetry es un conjunto de APIs, SDKs y herramientas estándar para **generar, recolectar y transportar** telemetría. Es la capa de transporte, **no almacena** ni reemplaza a Prometheus, Elasticsearch o Jaeger.

- **SDK:** instrumenta la aplicación con una API neutral (duraciones, solicitudes, errores, atributos de negocio), desacoplada del backend.
- **OTLP:** protocolo común para enviar métricas, logs y trazas, en lotes.
- **Collector:** proceso aparte que recibe, filtra, transforma, aplica sampling y reenvía a uno o varios destinos.

Instrumentación en dos capas: automática (infraestructura: HTTP, gRPC, tiempos; arma el esqueleto de la comunicación) y manual (dominio: tenant_id, método de pago, tamaño del carrito). La primera muestra por dónde pasó la solicitud; la segunda, qué estaba haciendo.

Correlación automática: el SDK mantiene trace_id y span_id en el contexto activo y los agrega a cada log, que se asocia a su traza por igualdad exacta de IDs.

Exemplars: muestra representativa adjunta a un punto de una métrica con el trace_id de una solicitud concreta (del p99 = 2.500 ms a una traza en Grafana). Lleva a un ejemplo, no prueba que sea la causa raíz.

Análisis diferencial multidimensional: comparar la distribución de atributos entre solicitudes exitosas y fallidas (zona A aparece mucho más entre las fallidas) para encontrar patrones no anticipados. Señala sospechosos, no garantiza la causa.

Logs estructurados y wide events

- **Logs de texto libre:** pensados para personas; extraer datos con regex es frágil.
- **Logs estructurados:** pares clave-valor consultables (service_name = payments AND severity_text = ERROR). Campos recomendados: timestamp, service_name, service_version, trace_id, span_id, severity_text y atributos.
- **Wide event:** evento estructurado rico en atributos que resume una **unit of work** (procesar una compra, registrar un usuario). Se acumula información durante la ejecución y se emite un único evento al final (ítems, total, método y proveedor de pago, región, duración, estado, tipo de error). Permite análisis diferencial sin reconstruir la operación desde muchas líneas. Todo wide event es un log estructurado, pero no al revés.

Métricas

Una métrica tiene nombre, etiquetas (labels) y valores en el tiempo. Son **preagregadas**: guardan el conteo, no el detalle, por eficiencia; lo no conservado no se puede reconstruir.

Las 4 golden signals (Google SRE):

- **Latencia:** separar exitosas de fallidas (si cae la base, la latencia baja porque responde errores rápido); mirarla junto a errores.
- **Tráfico:** demanda (RPS, sesiones, consultas por segundo). No es capacidad: se pueden recibir 2.000 req/s y procesar solo 1.500.
- **Errores:** fallas reales de la operación; no todo 4xx es falla del servicio.
- **Saturación:** qué tan lleno está el recurso más restringido. Se degrada antes del 100 %; la latencia creciente es señal temprana.

Percentiles: el promedio oculta experiencias muy malas. p50, p95 y p99 = valor bajo el cual queda ese porcentaje de solicitudes; el p99 muestra a los usuarios más perjudicados. La latencia tiene cola larga a la derecha: hay un mínimo físico, pero colas, bloqueos y red pueden alargarla mucho.

Cardinalidad: cada combinación de etiquetas es una serie temporal distinta; las series se multiplican por la cardinalidad de cada dimensión (explosión de cardinalidad) y consumen memoria en Prometheus. **Regla: alta cardinalidad (user_id, order_id) en spans y wide events, no en etiquetas de métricas;** en métricas solo etiquetas acotadas (http_method, status_code, endpoint).

Distributed tracing

- **Trace:** viaje completo de una solicitud. **Span:** operación individual delimitada en el tiempo (consulta SQL, llamada). El primero es el **root span**, sin padre.
- **Contenido de un span:** trace_id, span_id, parent_span_id, timestamp de inicio y duración.
- **DAG:** el trace es un grafo dirigido acíclico; nodos = spans, aristas = relaciones causales.
- **Línea de tiempo:** no sumar duraciones, porque los spans se anidan o corren en paralelo (los 450 ms de Payments ya están dentro de los 800 ms de Orders).
- **SpanKind:** CLIENT (llamada síncrona saliente), SERVER (atención de una entrante), PRODUCER (publica un mensaje asíncrono), CONSUMER (lo procesa; el productor no espera), INTERNAL (operación interna).
- **Relación con Lamport:** parent_span_id conserva un vínculo causal explícito sin depender de relojes. No es un reloj lógico, y padre e hijo normalmente se superponen en el tiempo.
- **Propagación in-band:** el contexto viaja dentro de los mensajes (cabecera HTTP *traceparent*); si un servicio no la propaga, se corta la correlación. **Out-of-band:** los spans se envían al collector por un camino aparte, en buffers y de forma asíncrona, para no sumar latencia.
- **Span links:** asocian spans relacionados sin forzar una jerarquía padre-hijo.

Sampling	Cuándo decide	Ventaja	Desventaja
Head-based	Al inicio (ej. 10 %)	Eficiente, reduce volumen desde el principio	Puede descartar una traza que después falla
Tail-based	Al terminar (100 % de errores y lentas, 5 % del resto)	Conserva las trazas más útiles	Hay que retener datos antes de decidir

Camino de diagnóstico: métrica (degradación global) → exemplar (solicitud concreta) → traza (dónde se fue el tiempo) → log o **profile** (consumo de recursos por función).

Debugging en producción

La causa raíz no está necesariamente donde aparece el error. **Falla en cascada típica:**

1. La base se degrada por **contención de bloqueos**: no cae, pero responde muy lento.
2. Orders agota threads y conexiones esperando, sin bug propio.
3. Los clientes hacen timeout y **reintentan**, amplificando el tráfico en un ciclo de retroalimentación.
4. El API Gateway devuelve **504**. Reiniciar el gateway no arregla nada: hay que reconstruir la cadena.

Core Analysis Loop:

1. **Observar** la anomalía agregada en métricas (el p99 sube desde las 10:15).
2. **Diferenciar dimensiones** con soft context (us-east-1a en el 98 % de las lentas vs. 15 % de las normales).
3. **Confirmar con trazas** individuales (la consulta a la base tardó 3.300 ms) y repetir con otras trazas.
4. **Repetir** sobre el subconjunto más chico hasta tener una explicación respaldada por evidencia.

Primero observar, después buscar patrones y finalmente investigar la causalidad, sin necesidad de saber de antemano qué preguntar.

Clase 11 — Resiliencia: timeouts, retries, circuit breakers, bulkheads y backpressure

Pregunta central: ¿cómo evitar que la falla de un componente tumbe todo? Si Payments deja de responder, Orders espera, acumula solicitudes, agota recursos y también cae.

Resiliencia distribuida: capacidad de seguir funcionando ante fallas parciales, aunque sea con capacidad reducida. No es que nada falle: es contener las fallas, evitar que se propaguen, poder recuperarse y mantener lo que no depende del componente caído (sin pagos, seguir mostrando productos).

Conceptos

- **Fault → error → failure:** el *fault* es la condición que origina el estado incorrecto (quizás invisible); el *error*, el comportamiento incorrecto observable (excepciones por falta de conexiones); la *failure*, cuando el sistema deja de cumplir su servicio (ninguna solicitud consigue conexión).
- **La falla depende del observador:** un proceso vivo con CPU baja puede estar totalmente bloqueado con hilos esperando E/S.

- **Propagación ("la grieta"):** se da por acoplamiento; más dependencias, más caminos. Un **failure mode** describe la cadena completa: origen, propagación y consecuencias.
- **Crackstoppers:** timeouts (no esperar indefinidamente), retries (recuperar fallas transitorias), circuit breakers (dejar de llamar a una dependencia enferma), fallbacks (respuesta alternativa), bulkheads (aislar recursos), backpressure y load shedding (controlar sobrecarga).
- **Disponibilidad** (operativo cuando se lo necesita) vs. **tolerancia a fallas** (seguir prestando servicio pese a ciertas fallas, p. ej. con redundancia) vs. **resiliencia** (contener el impacto, mantener lo posible y facilitar la recuperación).

Anatomía de una falla en cascada

- **Puntos de integración:** cada comunicación entre componentes es una dependencia.
- **Fan-out alto:** un pedido usa muchas dependencias y basta que una falle. **Fan-in alto:** muchos dependen de uno y su caída afecta a todos.
- **Falla lenta peor que falla rápida:** una dependencia caída que rechaza al instante libera el hilo; una viva pero lenta lo retiene.
- **Agotamiento del pool:** con hilos esperando red, la CPU puede estar casi ociosa y el servicio inutilizable. La saturación se mide sobre el recurso que se agota.
- **Reacciones en cadena:** si cae una instancia, el balanceador reparte su carga y sobrecarga a las otras.
- **Capacidades desbalanceadas:** un frontend grande puede ahogar a un backend chico; la capacidad del sistema la fija su cuello de botella.

Ley de Little: $L = \lambda \cdot W$ (L = solicitudes concurrentes, λ = tasa de llegada, W = tiempo en el sistema; vale en sistemas estables). Con 8 hilos y una dependencia que tarda 10 s, el throughput máximo es 8 / 10 = 0,8 pedidos/s; si llegan 5/s, el resto se acumula o se rechaza. Sin aumentar el tráfico, solo por la lentitud de una dependencia, el servicio se satura.

Tiempo en cola vs. procesamiento: si una solicitud espera 8 s en cola y se procesa en 200 ms, el servidor reporta 200 ms y el usuario sufre 8,2 s. Hay que medir desde el cliente.

Timeouts

Límite de espera antes de dar por fallida una operación: evita hilos bloqueados, acota el daño de una dependencia lenta y libera recursos. Ojo: que el cliente deje de esperar no detiene al servidor, que puede seguir trabajando para nadie.

- **Connect timeout:** cuánto esperar para establecer la conexión.
- **Read timeout:** cuánto esperar entre datos recibidos; no limita la operación completa (con read de 2 s una respuesta puede tardar 5,5 s llegando de a pedazos).
- **Deadline:** límite global de toda la operación.

- **Cómo elegirlo:** no la media (genera errores innecesarios) ni el máximo (reacciona tarde): **p99 de latencia normal + margen**.
- **Cadenas A → B → C:** el p99 de B medido desde A ya incluye a C en condiciones normales, pero no el peor caso con reintentos. Peor caso de B = $(R + 1) \cdot T + \Sigma e_i$; con T = 500 ms, R = 2 y esperas de 100 y 200 ms: $3 \cdot 500 + 300 = 1.800$ ms. Si A ya abandonó, B desperdicia recursos. **Regla:** los timeouts internos y sus reintentos deben caber en el plazo de la operación externa (presupuesto de tiempo extremo a extremo).
- **Timeout vs. fail fast:** el timeout acota la espera de algo ya iniciado; fail fast rechaza al instante lo que se sabe que no se podrá completar (pool lleno). Ninguno arregla la dependencia: para eso hacen falta retries, circuit breaker y fallback.

Retries

Reintentar sirve ante **fallas transitorias**, no persistentes. Dos condiciones: la operación es **idempotente** (clave de idempotencia para no cobrar dos veces) y la falla es **reintentable**.

- **Riesgo:** agregan carga justo cuando el servidor tiene menos capacidad: más carga → más lentitud → más timeouts → más reintentos, un ciclo que puede seguir aunque baje el tráfico original.
- **Fan-out:** con 5 llamadas que fallan con p = 0,1, el pedido sale bien solo $0{,}9^5 \approx 59$ % de las veces; con un reintento por llamada (fallas independientes) cada una falla 0,01 y el pedido sale bien ≈ 95 %. Costo: E[intentos] = 1 + p = 1,1 (≈10 % más tráfico); pero ante una caída total cada llamada hace 2 intentos y la carga se duplica justo cuando el servidor falla.
- **Reintentos en múltiples capas:** se multiplican entre sí. Hay que decidir en qué capa reintentar y limitar intentos totales y tiempo.
- **Backoff exponencial:** esperar entre intentos, creciendo: $d_n = \min(\text{tope}, \text{base} \cdot m^n)$, con m normalmente 2 y n desde 0.
- **Jitter:** componente aleatorio para que miles de clientes no reintenten sincronizados (**thundering herd / dogpile**). *Full jitter:* $e \sim U(0, d)$, dispersa mucho pero algunos reintentan casi enseguida. *Equal jitter:* $e = d/2 + U(0, d/2)$, garantiza una espera mínima pero dispersa menos.
- **Retry budget:** límite por pedido (ej. 3 intentos) y límite global por cliente (ej. reintentos ⩽ 10 % de las solicitudes originales). Con pocas fallas se reintenta; con muchas se agota el presupuesto y se deja de insistir.
- **Conclusión del experimento de la clase:** los reintentos mejoraron mucho el éxito ante errores transitorios; más reintentos suben el éxito pero también trabajo y tiempo; el jitter no mostró ventaja porque había un solo cliente.

Circuit breaker

Deja de enviar solicitudes a una dependencia que falla demasiado: evita llamadas condenadas, libera recursos propios, alivia a la dependencia, le da tiempo a recuperarse y corta la propagación. Un retry insiste; el breaker evita intentar.

Estado	Comportamiento	Transición
CLOSED	Las llamadas pasan y se cuentan las fallas	Si se supera el umbral → OPEN
OPEN	Rechaza sin llamar durante un tiempo configurado	Al vencer el tiempo → HALF_OPEN
HALF_OPEN	Deja pasar pocas solicitudes de prueba	Éxito → CLOSED; falla → OPEN

- **Qué cuenta como falla:** errores que indican dependencia enferma (5xx, timeouts); **no los 4xx**, que pueden ser culpa del cliente.
- **Necesita timeout:** sin él no registra la falla lenta. El timeout decide cuándo dejar de esperar; el breaker, si conviene dejar de llamar.
- **Política por fallas consecutivas:** abrir tras k fallas seguidas responde a rachas, no a la tasa real. Con 10 % de error y k = 3 aparece una racha cada ≈1.110 solicitudes (a 100 req/s, cada ≈11 s), abriendo seguido sin motivo; subir k a 10 casi nunca abre pero tarda mucho en detectar un 50 % de error. A más tráfico, más rachas por unidad de tiempo.
- **Política por tasa en ventana:** mira las solicitudes recientes con un mínimo de llamadas (ej. 5) antes de decidir y un umbral (ej. 50 %). Distingue errores ocasionales de una dependencia realmente enferma. El tamaño de ventana depende del tráfico, la variabilidad y la velocidad de detección necesaria.
- **Tiempo en OPEN:** muy corto prueba contra algo no recuperado; muy largo rechaza aunque ya se recuperó.
- **Pruebas en HALF_OPEN:** pocas (clásico: una) para no volver a saturar; se puede exigir más de un éxito.
- **Granularidad:** un breaker por dependencia y por proceso (si falla Payments, seguir llamando a Stock); su estado es local a cada instancia.
- **Con retries:** cada intento real cuenta y vuelve a pasar por el breaker; un rechazo por breaker abierto **no se reintenta**.
- **Implementación:** thread-safe, con el lock protegiendo solo el estado del breaker (no la llamada de red, que serializaría todo), y con sus cambios de estado registrados y monitoreados.

Graceful degradation y fallbacks

El breaker protege recursos; el fallback decide si el usuario igual recibe algo útil. **Graceful degradation:** seguir dando una respuesta útil aunque falte una funcionalidad (la tienda funciona sin recomendaciones). **Fallback:** estrategia alternativa: caché, respuesta genérica (productos populares), servicio secundario u omitir la funcionalidad, indicándolo (campo *degraded*).

Una buena degradación es **explícita** (quien la recibe lo sabe; no se vuelve error silencioso), **definida de antemano** con el negocio y **más barata que el camino normal**. Solo conviene si la respuesta alternativa sigue siendo correcta y aceptable.

Bulkheads

Dividir recursos en compartimientos aislados para que una parte no agote los de las demás. Ejemplo: Generator con un pool compartido de 8 hilos y cuatro dependencias; si *special* se vuelve lenta, bloquea los 8 hilos y caen también upper, lower y digit. Con un límite por dependencia, *special* solo agota los suyos. No elimina la dependencia funcional: lo que necesita *special* igual falla.

Se pueden aislar hilos, CPU, instancias, clústeres y zonas o regiones. Caso especial: **reservar recursos de administración** para poder diagnosticar un servicio saturado. **Costo:** capacidad ociosa que otra partición no puede usar; se cambia eficiencia por aislamiento, buscando límites naturales según criticidad.

Backpressure y load shedding

Una **cola ilimitada** (procesar 100/s recibiendo 150/s) produce respuestas cada vez más lentas, memoria creciente, timeouts y reintentos. Mejor colas acotadas. Con la cola llena se puede: fingir aceptar y descartar (pérdida silenciosa), descartar otra (la más vieja), rechazar la nueva o frenar al productor.

- **Backpressure:** el consumidor presiona hacia atrás para que el productor baje su ritmo. Funciona con productores conocidos y controlables (servicios internos), no con miles de clientes de una API pública.
- **Load shedding:** rechazar rápido la carga que no se podrá atender a tiempo, en lugar de aceptarla y fallar por timeout. Típico en el borde del sistema.
- Analogía: backpressure = el encargado pide a los mozos que no manden comandas hasta que la cocina pueda; load shedding = el restaurante deja de tomar pedidos. Se usan juntos.

Caso Dressy: el balanceador interpretó los rechazos baratos del load shedding como eficiencia y mandó más tráfico a la región saturada. Corrección: tratar un rechazo como señal de sobrecarga (costo equivalente a CPU > 100 %). Dos mecanismos correctos por

separado pueden interactuar mal.

Orden de combinación

1. **Bulkhead:** ¿hay capacidad en la partición? Si no, ni intentar.
2. **Circuit breaker:** ¿la dependencia está sana? Si está abierto, no llamar.
3. **Timeout:** intento con tiempo acotado.
4. **Retry:** si corresponde, con backoff + jitter, volviendo a pasar por el breaker.
5. **Fallback:** respuesta alternativa o error.

Un servicio puede estar vivo, con CPU baja y respondiendo el health check, y aun así fallar para sus usuarios.

Clase 12 — SLOs, error budgets, chaos engineering, incidentes y postmortems

Pregunta central: "tenemos 99,9 % de disponibilidad", ¿es bueno o malo? No se puede responder sin un objetivo.

SLI, SLO y SLA

- **SLI (indicador):** métrica de calidad del servicio: disponibilidad, latencia, durabilidad, frescura, tasa de errores. SLI = eventos buenos / eventos válidos × 100, donde "bueno" es lo que satisface al usuario. Normalmente los 4xx atribuibles al cliente no cuentan como falla y los 5xx sí.
- **SLO (objetivo):** meta para un SLI en una **ventana temporal** (⩾ 99,9 % de solicitudes correctas en los últimos 30 días). Ventana **fija** (1 al 31 de octubre: el presupuesto se reinicia de golpe) o **deslizante** (últimos 30 días: las fallas salen de a poco; más representativa para operar).
- **SLA (acuerdo):** compromiso con el cliente con **consecuencias contractuales** (SLA 99,9 % mensual con descuento si no se cumple). El SLO interno se pone más exigente (99,95 %) para reaccionar antes de incumplir el SLA.
- **No se busca 100 %:** cada nueve extra cuesta cada vez más esfuerzo que podría ir a funcionalidades, y el usuario muchas veces no lo nota. La confiabilidad es una decisión de gestión de riesgo.

Error budget y burn rate

Error budget = 1 − SLO: con SLO 0,999 se tolera 0,1 % de solicitudes fallidas en la ventana. No es una meta de fallas sino el riesgo tolerable; sirve para decidir cuánto riesgo asumir al desarrollar y operar. Que un mecanismo de resiliencia mejore algo no significa que la

mejora alcance.

Presupuesto restante	Política de despliegue
Más del 50 %	Despliegues normales, con canary y rollback automático
Entre 50 % y 0 %	Más conservadores: graduales y en horarios de poco tráfico
0 %	Congelar todo cambio que no sea de confiabilidad o seguridad

Además, todo incidente que consuma más del 20 % del presupuesto requiere postmortem.

Burn rate = tasa de error observada / tasa de error permitida (1 − SLO). BR = 10 consume el presupuesto 10 veces más rápido de lo permitido; **tiempo de agotamiento = T / BR** (T = ventana del SLO). Con ventana deslizante el presupuesto también se recupera a medida que salen fallas viejas.

Alertas sobre el SLO

Alertar por CPU, memoria o disco señala causas posibles, no impacto en usuarios, y produce **alert fatigue**: se ignoran las alertas y se pierde la importante. Una alerta útil suena solo si hay un problema real para el usuario y quien la recibe puede actuar.

- **Alertar al incumplir el SLO:** demasiado tarde.
- **Alertar cuando queda poco presupuesto (ej. 30 %):** no distingue gastar 70 % en tres semanas de gastarlo en una hora.
- **Alertar por velocidad de consumo (burn rate):** distingue una degradación leve de una falla crítica.

Multi-window, multi-burn-rate: una **ventana larga** confirma que el consumo es significativo y una **corta** que sigue ocurriendo ahora (si la falla terminó hace 10 minutos, la ventana de 5 min ya está sana y la alerta se apaga). **Page** = despertar a la guardia; **ticket** = puede esperar al horario laboral. Configuración típica (SRE) para SLO a 30 días:

Escenario	Ventanas	Burn rate	Presupuesto consumido	Acción
Falla rápida y grave	1 h y 5 min	14,4	2 %	Page
Degradación intermedia sostenida	6 h y 30 min	6	5 %	Page

Escenario	Ventanas	Burn rate	Presupuesto consumido	Acción
Degradación lenta de varios días	3 días y 6 h	1	10 %	Ticket

Origen del 14,4: T = 30 × 24 = 720 h y la proporción consumida es BR · t / T; para consumir 2 % en 1 h: BR = 0,02 × 720 / 1 = 14,4. Las alertas por proporciones necesitan volumen de datos suficiente para ser interpretables.

Chaos engineering

La seguridad no se compone: dos componentes que andan bien por separado pueden fallar combinados. Los tests unitarios aíslan, los de integración usan mocks y staging no reproduce instancias, tráfico ni red de producción.

Chaos engineering: experimentar con fallas controladas para comprobar el comportamiento ante condiciones adversas. **No es romper cosas al azar:** lleva hipótesis, comportamiento esperado, perturbación controlada, métricas y condiciones de seguridad y recuperación (ej. Chaos Monkey: si cae una réplica, las otras dos deben seguir respondiendo).

1. **Definir el steady state:** comportamiento normal medible (baseline, a partir de los SLI).
2. **Formular una hipótesis concreta y medible:** “si especiales se vuelve lento, las solicitudes afectadas fallan rápido y las demás siguen normales” (no “ver si aguanta”).
3. **Inyectar una falla realista y controlada:** latencia de 2 s, apagar una réplica, hacer fallar llamadas, pérdida de paquetes, particiones, CPU o disco.
4. **Intentar refutar la hipótesis:** si todas las solicitudes se vuelven lentas, hay una debilidad.

- **Grupo de control:** solicitudes sin falla para comparar, porque hay errores naturales aunque no haya experimento.
- **Falla rápida vs. lenta:** hay que probar ambas; la lenta retiene hilos y conexiones, acumula solicitudes y se propaga.
- **Blast radius:** alcance máximo de la perturbación. Precauciones: empezar con poco tráfico, tener observabilidad, criterio de aborto definido de antemano (ej. burn rate > 14,4 en la ventana de 5 min), plan de recuperación probado (detener la inyección no garantiza recuperarse sola) y progresar de local a integración a una fracción de producción.
- **Consume error budget:** con el presupuesto casi agotado, se posterga. Puede revelar dependencias ocultas.

- **Automatización:** repetir experimentos periódicamente para detectar regresiones, porque el sistema cambia. También se ensayan personas y procesos (DiRT de Google: Disaster Recovery Testing).

Gestión de incidentes

Incidente: evento que degrada o amenaza el servicio y requiere respuesta coordinada (no toda falla lo es). Se declara si hay que involucrar a otro equipo, si es visible para clientes, si lleva una hora de investigación sin resolverse o si saltó una alerta de burn rate tipo page. **Mejor declarar temprano.**

Sin gestión aparecen tres problemas: foco solo técnico (nadie mira el panorama ni la mitigación), mala comunicación (trabajo duplicado, información contradictoria) y cambios sin coordinación que interfieren entre sí.

Rol	Responsabilidad
Incident commander	Visión general, asignar tareas, priorizar, remover obstáculos; coordina, no necesariamente resuelve
Operaciones	Investiga y cambia el sistema: dashboards, logs, trazas, reinicios, revertir despliegues
Comunicación	Informa a interesados para que los ingenieros no se interrumpan
Planificación	Tareas de más largo plazo, relevos y seguimiento

Elementos de coordinación: **puesto de mando** (canal o videollamada única, que además deja registro), **documento vivo** (resumen, estado, roles, tareas, línea de tiempo) y **traspaso explícito** del rol con confirmación.

Fin del incidente: no es solo arreglar el bug, sino verificar con criterios de salida (ej. SLIs normales y estables durante 30 minutos).

Prioridades: detener el daño → restablecer el servicio → analizar causas (preservando evidencia). Primero mitigar, después investigar.

Runbook: guía operativa para responder a problemas conocidos. Breve, útil y actualizada; guía el razonamiento, no lo reemplaza ni cubre todo.

Postmortems

Documento posterior para entender qué pasó y definir acciones que reduzcan la probabilidad de que se repita. Resolver recupera el servicio; el postmortem mejora el sistema. No todo incidente lo requiere (política: > 20 % del error budget). Debe

entenderlo alguien que no estuvo.

Estructura: resumen (qué y cuánto duró), impacto (usuarios, errores, duración, presupuesto consumido), detección, disparador y causas, resolución, timeline, lecciones (qué funcionó, qué no y dónde hubo suerte, p. ej. que justo estuviera el único que conocía una dependencia), acciones y evidencia.

- **Blameless:** analizar condiciones y decisiones sin culpar personas; preguntar por qué el sistema permitió llegar ahí (ej. validar en CI/CD que toda dependencia tenga timeout).
- **Disparador vs. factores contribuyentes:** el disparador inicia el incidente; los factores (timeout mal configurado, falta de experimentos de latencia, malos dashboards, rollback difícil) permiten que se agrave y propague.
- **Buenas acciones:** concretas y verificables, con responsable, prioridad y seguimiento, y orientadas a cambiar el sistema. Son lo más valioso del postmortem.
- **Prevenir** (que no vuelva a pasar: validaciones automáticas) vs. **mitigar** (reducir el impacto si pasa: rollback fácil, mejor aislamiento).

Ideas transversales y distinciones que más se confunden

Todo el curso repite una misma lógica: **ningún mecanismo elimina el costo, lo desplaza**, y cada decisión se defiende explicitando falla asumida, garantía y trade-off (D = P, F, G, S, T).

No confundir	Con	Diferencia clave
Timeout	Falla	El timeout solo dice que no llegó respuesta a tiempo
Escrito / replicado	Durable	Durable es siempre “frente a qué falla”
Committed	Confirmado al cliente	El ACK sale después del commit durable (WAL, Raft)
Lamport	Vector clocks	Lamport no distingue causalidad de concurrencia; los vectoriales sí
Detectar un conflicto	Resolverlo	Los vector clocks detectan; resolver depende del negocio
R + W > N	Consenso	Intersección no da orden total ni versión autoritativa
Serializabilidad	Linearizabilidad	La primera no exige orden real; consistencia externa = ambas
Snapshot Isolation	Serializable	SI permite write skew

No confundir	Con	Diferencia clave
MVCC	Serializabilidad	MVCC administra versiones; el aislamiento define qué se permite
Outbox	Inbox	Outbox no pierde la publicación; inbox no duplica efectos; ninguna evita duplicados de entrega
Committed offset	Efecto aplicado	El offset es progreso de lectura, no prueba de efecto
Exactly-once	Propiedad de la red	Solo vale dentro de una frontera coordinada
Compensación	Rollback	La compensación es una acción nueva que también puede fallar
Rollback de versión	Deshacer efectos	Volver el software atrás no anula pagos ni eventos
Separar en servicios	Autonomía	Un monolito distribuido paga costos sin ganar autonomía
Monitoreo	Observabilidad	Known-unknowns vs. unknown-unknowns
Uptime	Disponibilidad	Un 503 rápido no es disponibilidad de la operación
Retry	Circuit breaker	Uno insiste; el otro evita intentar
Backpressure	Load shedding	Frenar al productor vs. rechazar carga
SLO	SLA	El SLA tiene consecuencias contractuales
Disparador	Causa	Los factores contribuyentes permiten que el disparador escale